

提出日: 2026 年 5 月 13 日

第 4 回ハッカソン議事録

開催日時: 2026 年 5 月 13 日 12:00~16:00

開催場所: 津田沼校舎 7 号館 3F 732 講義室

班名: 23 班

メンバー: 24G1038 片岡聖

24G1131 地曳賢人

24G1131 御代川稜 (議事録作成者)

25G1021 梅澤颯太

25G1053 齋藤翔太

25G1065 塩澤匠生

欠席者: なし

目的: チーム全体のプロジェクトの計画書とシステムの設計書の議論をする。
来週のプレゼンテーションをできるようにする。

1 議題一覧

- チーム全体のプロジェクトの計画書の議論
- システムの設計書の議論.
- プレゼンテーションについての議論する

2 討議内容

2-1. 題名決め

塩澤: たくさん案をだしてみよう, 「エアタクト」, 「エアケストラ」, 「タクトーン」, 「タクトビート」
とが出た.

地曳: タイトル見ただけでわかるようにしたい.

齋藤: 去年はどんな感じでやったんですか?

片岡: 堅く文章の説明の班もあったし, ユーモアあふれるグループもあったよ.

梅澤: メインタイトルは「タクトーン」でサブタイトルを IMU ジェスチャーで奏でる Arduino オーケストラ はどうでしょう?

御代川: いいと思う.

3 TA さんとの会話

TA さんから「指揮棒の振りの大小を検知するのむずくない. 対応するか?」3 軸加速度センサを一軸としてとって計算にして, 合計値で考える. そのため, 角度は関係なく, 距離などで 10cm 振られないと意味無くする. 加速度が止まったのを確認してからフラグを落とす, つまり距離やタイミングで調整する. 振り後の距離と振り始め検出と振り終わり検出で決めてく.

TA さんから「実際に振るのが, 小さく降ったら音を小さく, 大きく振ったら大きく音にはできる?」回答は, 小さいと平均的に算出して, 振りの直前の振りの何%振らせないと起動しないなどの

対策をする予定。

3-1. スライドの担当決め

技術的・社会的な課題と既存システム: 御代川稜

提案システム 1: 塩澤

提案システム 2: 梅澤

提案システム 3: 斎藤

到達目標: 地曳

役割分担: 片岡

必要な機材: 塩澤

システム構築スケジュール: 地曳

その他: 片岡

まとめ: 御代川

3-2. 個人毎の次回までに実施する内容

片岡聖・担当箇所のスライド作成・役割分担のスライド作成・その他の担当をする

地曳賢人・担当箇所のスライド作成・システム構築スケジュールと到達目標のスライド作成

御代川稜・担当箇所のスライド作成・システム構築スケジュールと到達目標のスケジュールの作成と
まとめのスライド作成

梅澤颯太・担当箇所のスライド作成・提案システム 2 のスライド作成

斎藤翔太・担当箇所のスライド作成・提案システム 3 のスライド作成

塩澤匠生・担当箇所のスライド作成・提案システムと必要な機材のスライド作成

4 決定事項

・MOP, V&V, TPM の確定・ガントチャートの修正・ゲーム性の作成をする。

5 継続審議事項・保留事項

・スライドの作成・ゲーム性の利用

6 次回の会議の日時・場所・予定議題

開催日時: 2026 年 5 月 20 日 12:00~16:00

開催場所: 津田沼校舎 7 号館 3F 732 講義室

予定議題: チーム全体のプロジェクトの計画書とシステムの設計書の修正
及び、プレゼンテーションの振り返り

7 その他, 特記事項

特になし

8 プロジェクトの計画書の原案

今回会議の必要事項のプロジェクトの計画書の原案は以下のようになる。

9 プロジェクトの概要

本プロジェクトでは、13 週をかけてハードウェアとソフトウェアを組み合わせたシステムを Arduino を用いてオーケストラを目標とする。23 班では指揮によるテンポ変化システムを開発する。その際に、テンポや音量を変化できるようにする。また、指揮者がどのくらいの正確性があるか指揮をできているかのゲームをできるようにする。

9-1. プロジェクトの題目

「タクトーン IMU ジェスチャーで奏でる Arduino オーケストラ」

9-2. プロジェクトの目的

指揮者付きのオーケストラを実際にやろうとすると 1 人ではできない。そのため、指揮者の方の練習の機会が少なくなってしまう。また、楽器は金属や木製であるため水に触れると腐食してしまう。そのため、本プロジェクトの目的は、時間帯や天候、ネットワーク環境などの外部条件に左右されず、明るさなどの制約も受けない環境で、音楽経験のない人でも直感的な指揮動作だけで演奏に参加できるシステムを開発することである。具体的には、指揮棒を振る周期を検出して楽曲のテンポへ反映し、さらに加速度の大きさに応じた音量変化や LED による視覚的演出を加えることで、誰でも簡単に楽しめる体験型電子オーケストラの実現を目指す。また、指揮棒の練習として正確に指揮棒を振れているかをゲーム性をもたらし視覚的に上達を促すようにする。最終的なゴールは、指揮棒を振った際に、振りの大きさや周期の速度によって音楽の音量の大きさやテンポが変化したならば完成である。課題としては以下のような観点から考えられる。

- 指揮動作を正確に検出する課題がある。なぜなら、指揮棒の振りには個人差があり、速さ・大きさ・角度も異なるため、さまざまな振り方でも安定して動作を検出する必要がある。
- テンポヘリアルタイム反映する課題がある。なぜなら、指揮棒を振った周期に合わせて、音楽のテンポを遅延なく自然に変化させる必要がある。反応が遅いと操作感が悪くなる。
- 誤検出を防ぐ課題がある。なぜなら、小さな揺れや不要な動きを誤って指揮動作と認識すると、テンポが乱れるため、必要な動作だけを判定する仕組みが必要である。
- 音楽経験がない人でも使いやすくする課題がある。なぜなら、専門知識がなくても直感的に使えるように、簡単な操作方法と分かりやすいフィードバックが必要である。
- 視覚的に状態を伝える課題がある。なぜなら、現在のテンポや音量変化が分かりにくいと利用者が操作しづらいため、LED 表示などで演奏状態を可視化する必要がある。
- 環境条件に左右されない課題がある。なぜなら、明るさ、時間帯、天候、通信環境などに依存せず、屋内外で安定して使用できるシステムにする必要がある。

9-3. 既存の技術とプロジェクトの独自性

既存技術として、テルミンのような非接触型電子楽器や、カメラ・センサを用いて身体動作を音楽表現へ変換するシステムが存在する [?]。また、Wii Music[?] では、コントローラの動作から拍を検出し、音楽のテンポを制御している。

しかし、これらのシステムは高い演奏技術を必要としたり、明暗などの環境条件に影響されやすいという課題がある。さらに、多くは 1 台の機器内で音源生成を行う構成であり、複数端末による分散演奏や、リアルタイムな指揮制御には十分対応していない。

以上より、複数マイコンによる分散演奏と、身体動作によるリアルタイム指揮制御を両立するシス

テムには新規性がある。

10 スコープ・マネジメント

10-1. 成果物

本プロジェクトにおける成果物を以下に示す。

- Arduino R4 WiFi UNO を用いたオーケストラの実施
- 4 種類の音色を生成するプログラム
- 指揮棒を用いたゲーム
- システム構成図
- 動作確認結果および評価資料
- プロジェクト計画書および最終報告書

10-2. プロジェクトの対象範囲と非対象範囲

10-2.1 プロジェクトの対象範囲

本プロジェクトの対象範囲を以下に示す。

- Arduino を用いた指揮動作検出システムの開発
- 加速度センサを用いた指揮棒の振り動作の取得
- 指揮棒の振り周期に応じた音楽テンポ制御機能
- エッジ検出によるテンポ変化機能
- 指揮棒の振り速度に応じた音量変化機能
- ハードウェアとソフトウェアを組み合わせたシステム統合
- システムの動作確認および性能評価

10-2.2 プロジェクトの非対象範囲

本プロジェクトでは以下の内容を対象外とする。

- プロオーケストラレベルの高精度な指揮解析
- AI を用いた自動指揮認識機能
- 複数人による同時指揮への対応
- 市販音楽制作ソフトウェアとの連携
- 楽曲の自動生成機能
- 複雑なジェスチャ認識機能

10-3. 機能分解構造 FBS

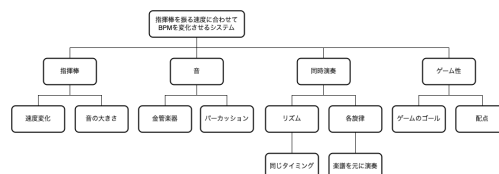


図 1 本プロジェクトの FBS 図

本プロジェクトの FBS を図 1 に示す。FBS では、「指揮棒を振る速度に合わせて BPM を変化させるシステム」に必要な機能を階層的に整理した。指揮棒、音、同時演奏、ゲーム性の 4 つを主要機能として分類し、さらに速度変化、音の大きさ制御、金管楽器音生成、パーカッション生成、リズム制御、各旋律の同期演奏、ゲームのゴール、配点などへ細分化した。また、リズムには同じタイミングや各旋律には楽譜を元に演奏と細分化している。これにより、本システムが実現すべき動作や機能要件を明確化した。

10-4. 製品分解構造 PBS

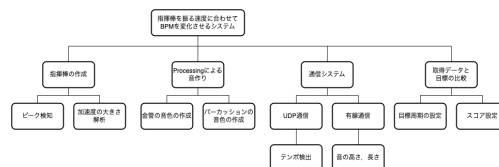


図 2 本プロジェクトの PBS 図

本プロジェクトの PBS を図 2 に示す。PBS では、「指揮棒を振る速度に合わせて BPM を変化させるシステム」を実現するために必要な成果物を階層的に整理した。指揮棒の作成、Processing による音作り、通信システム、取得データと目標の比較の 4 つを大項目として分類し、さらにピーク検知、加速度解析、金管楽器音色生成、パーカッション音色生成、UDP 通信、有線通信、目標周期の設定、スコア設定などの構成要素へ細分化した。また、UDP 通信にはテンポ検出、有線通信には音の高さや長さなどと細分化した。これにより、本システムを構築するために必要なハードウェアおよびソフトウェアの成果物を明確化した。

10-5. FBS と PBS の対応関係

PBS と FBS の対応関係として、PBS はシステムを構成する成果物や構成要素を示しているのに対し、FBS はそれら成果物によって実現される機能を示している。例えば、PBS における「加速度の大きさ解析」は、FBS における「速度変化」や「音の大きさ」の機能実現に対応している。また、PBS の「金管の音色の作成」および「パーカッションの音色の作成」は、FBS の「金管楽器」および「パーカッション」の機能に対応している。さらに、PBS の「UDP 通信」や「有線通信」は、FBS の「同時タイミング」や「各旋律」の同期演奏機能を実現するために用いられる。さらに、PBS の「目標周期の設定」や「スコア設定」は、FBS の「ゲームのゴール」や「配点」の機能を実現するために用いられる。また、PBS の「テンポ検出」や「音の高さや長さ」は、FBS の「同じタイミング」や「楽譜を元に演奏」のゲーム性機能を実現するために用いられる。

このように、PBS で整理した成果物を組み合わせることで、FBS で定義した各機能を実現する構成となっている。

11 作業計画

11-1. 作業分解構造 WBS と管理番号

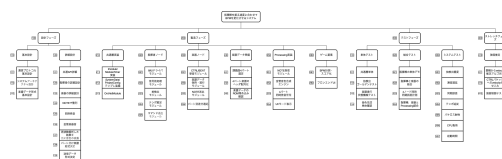


図3 本プロジェクトのハードウェアの構成図

図3は本プロジェクトのWBS図である。また、管理番号も記載してある。最終的な役割分担、またそれぞれの工程でどれほど時間がかかるかの表は以下の表1、表2、表3に記載する。

表1 新たに作成した作業分担表

時間的フェーズ	番号	要素成果物	活動タスク	担当者	必要時間
100 設計フェーズ (3週間)	110	基本設計	111 FBS の最終化	全員	2日間
			112 PBS の最終化	全員	2日間
			113 MOE/MOP/TPM の最終化	全員	2日間
			114 システムアーキテクチャ設計	塩澤	1週間
			115 通信プロトコル基本設計	塩澤	1週間
			116 楽譜データ形式基本設計	齊藤	1週間
			117 音色合成基本設計	梅澤	1週間
	120	詳細設計	121 共通層 API 詳細	塩澤	1週間
			122 指揮者の詳細設計	塩澤	1週間
			123 楽器の詳細設計	塩澤	1週間
			124 Processing 詳細設計 NOTE で整列	梅澤	1週間
			125 Processing 詳細設計 同時発音	梅澤	1週間
			126 Processing 詳細設計 金管楽器 音	梅澤	1週間
			127 Python による周波数分析	齊藤	1週間
			128 楽譜データ詳細設計 4パート 分け, 楽譜形式決定	斎藤	1週間
			129 楽譜データ詳細設計 Ar- duino PC 間送信データ決定	斎藤	1週間

表 2 新たに作成した作業分担表

時間的フェーズ	番号	要素成果物	活動タスク	担当者	必要時間
200 製造フェーズ (2 週間)	210	共通層実装	211 IModule/ModuleTimer 実装	塩澤	2 日間
			212 SystemData/ProjectConfig テンプレ整備	塩澤	1 日間
			213 OrcNetModule	塩澤	3 日間
	220	指揮者	221 IMU ドライバモジュール	塩澤	2 日間
			222 信号前処理モジュール	塩澤	2 日間
			223 拍検出モジュール	塩澤	3 日間
			224 テンポ推定モジュール	塩澤	1 日間
			225 コマンド送出モジュール	塩澤	1 日間
	230	楽器	231 CTRL/BEAT 受信モジュール	塩澤	2 日間
			232 楽譜データ保持・進行ロジック	塩澤	2 日間
			233 NOTE 送出モジュール	塩澤	1 日間
			234 パート別差分適用	塩澤	1 日間
	240	楽譜準備	241 課題曲 4 パートの選定	齊藤	1 日間
			242 4 パート楽譜のヘッダ配列化	齊藤	18 日間
			243 楽譜データの ROM 埋め込み確認	齊藤	2 日間
	250	Processing 実装	251 NOTE 受信モジュール	梅澤	1 日間
			252 金管音色合成エンジン	梅澤	18 日間
			253 4 パート同時発音管理	梅澤	5 日間
			254 UI/モード表示	梅澤	1 日間
	260	ゲーム実装	261 BPM 計測	梅澤	3 日間
			262 フロントエンド UI	梅澤	4 日間

11-2. アローダイアグラムとクリティカルパス

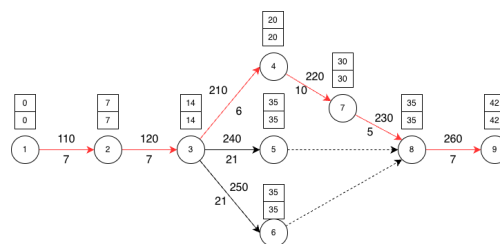


図 4 本プロジェクトのアローダイアグラムとクリティカルパス付きの図

表 1, 表 3 と図 5 より図 4 は, アローダイアグラムとクリティカルパス付きの図である。

図 4 より, 110「基本設計」を実施した後に, 120「詳細設計」を行う。その後, 設計内容を基にして, 210「共通層実装」, 240「楽譜データ準備」, 250「Processing 実装」を並列に進める構成とした。さらに, 共通層実装完了後に 220「指揮者ノード実装」を行い, 続いて 230「楽器ノード実装」を実施する。また, 240「楽譜データ準備」および 250「Processing 実装」は, 230「楽器ノード実装」と合

表 3 新たに作成した作業分担表の続き

時間的フェーズ	番号	要素成果物	活動タスク	担当者	必要時間
300 テストフェーズ (2 週間)	310	単体テスト	311 共通層単体	塩澤	2 週間
			312 拍検出ゴールデンテスト	塩澤	2 週間
			313 楽譜進行状態機械テスト	塩澤	2 週間
			314 音色生成単体確認	梅澤	2 週間
	320	結合テスト	321 指揮単体デモ	塩澤	2 週間
			322 指揮者と楽器の結合	塩澤	2 週間
			323 4 ノード同期誤差計測	全員	2 週間
			324 全ノードと Processing 結合	全員	2 週間
	330	システムテスト	331 拍検出精度	塩澤	2 週間
			332 通信遅延	塩澤	2 週間
			333 動機誤差	全員	2 週間
			334 テンポ追従	塩澤	2 週間
			335 パケロス耐性	塩澤	2 週間
			336 CPU 負荷	塩澤	2 週間
			337 起動時間	塩澤	2 週間
400 ストレッチフェーズ	410	強弱推定	411 velocity 推定アルゴ実装	塩澤	1 週間
			412 CTRL パケットへの velocity 乗せこみ	塩澤	1 週間
			413 強弱制御テスト	塩澤	1 週間

流した後、260「システム統合・テスト」へ進む構造となっている。最後に、全体の統合および動作確認が完了した後、プロジェクト終了となる。図中の赤線で示した経路はクリティカルパスである。この経路上の作業が遅延した場合、プロジェクト全体の完了時期へ直接影響を与えるため、重点的な進捗管理を行う必要がある。一方で、240「楽譜データ準備」および250「Processing 実装」には余裕時間が存在するため、他作業との並列実行によって効率的に開発を進める計画とした。

11-3. 役割分担

本プロジェクトでは、作業内容を「設計フェーズ」、「製造フェーズ」、「テストフェーズ」、「ストレッチフェーズ」の4段階に分類し、役割分担の表を作成した。具体的に表1、表2、表3の役割分担をした。設計フェーズでは、FBS、PBS、MOE/MOP/TPMの最終化を行い、プロジェクト全体の要求事項や構成を整理した。その後、システムアーキテクチャ設計、通信プロトコル設計、楽譜データ形式設計、音色合成設計などを実施した。また、共通層APIや指揮者ノード、楽器ノード、Processing側処理などの詳細設計を行い、後続の実装フェーズへ必要な仕様を明確化した。さらに、Pythonを用いた周波数分析により、金管楽器らしい音色生成を行うための基礎データを取得した。製造フェーズでは、共通層実装、指揮者ノード実装、楽器ノード実装、楽譜データ準備、Processing実装を並行して進めた。共通層では、IModuleやModuleTimerなどの基盤機能を実装し、ノード間で共通利用できる構造を整備した。指揮者ノードでは、IMUドライバ、信号前処理、拍検出、テンポ推定などを実装し、指揮棒の動きからテンポ情報を抽出できるようにした。楽器ノードでは、受信モジュールやNOTE送出モジュールを実装し、受信したテンポ情報に応じて楽譜を演奏する機能を実現した。また、Processing側ではNOTE受信モジュール、金管音色合成エンジン、4パート同時

発音管理を実装し、複数楽器による同時演奏を可能とした。加えて、ゲーム要素として BPM 計測機能やフロントエンド UI も実装した。テストフェーズでは、単体テスト、結合テスト、システムテストを段階的に実施する計画とした。単体テストでは、共通層や拍検出モジュール、音色生成モジュールなどの個別動作を確認する。結合テストでは、指揮者ノードと楽器ノードの連携動作、全ノードと Processing 間の通信動作を確認する。システムテストでは、拍検出精度、通信遅延、同期誤差、テンポ追従性能、パケットロス耐性、CPU 負荷などを測定し、本システム全体の性能評価を行う。ストレッチフェーズでは、velocity 推定アルゴリズムを用いた強弱制御機能の実装を行う計画とした。

11-4. ガントチャート

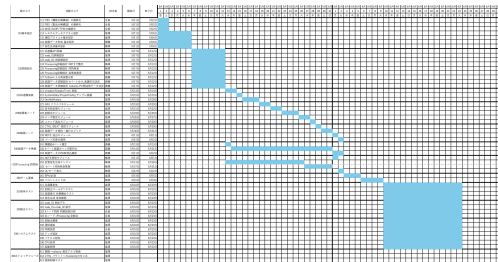


図5 本プロジェクトのガントチャート

図5は本プロジェクトのガントチャートである。本プロジェクトでは、全体を「設計フェーズ」、「製造フェーズ」、「テストフェーズ」、「ストレッチフェーズ」の4段階に分け、約2か月間のスケジュールで開発を進める計画とした。設計フェーズでは、FBS、PBS、MOE/MOP/TPMの最終化を行った後、システムアーキテクチャ設計、通信プロトコル設計、音色合成基本設計などを実施する。さらに、関数の詳細設計や楽譜データ形式設計、音色設計などを行い、後続の実装フェーズへ必要な仕様を確定する。製造フェーズでは、共通層実装、指揮者ノード実装、楽器ノード実装、楽譜データ準備、Processing側実装を並列に進める。特に、IMUドライバモジュール、拍検出モジュール、テンポ推定モジュールなどの指揮検出機能に加え、金管音色合成エンジンや4パート同時発音管理などの音響処理機能を重点的に実装する。また、楽譜データのROM埋め込み確認やNOTE送受信モジュールを実装することで、各ノード間通信による同期演奏を実現する。テストフェーズでは、単体テスト、結合テスト、システムテストを段階的に実施する。単体テストでは各モジュール単位での動作確認を行い、結合テストでは関数同士の同期動作や、全ノードとProcessing間の連携動作を確認する。さらにシステムテストでは、拍検出精度、通信遅延、同期誤差、テンポ追従性能、パケットロス耐性、CPU負荷などを測定し、本システム全体の性能評価を行う。ストレッチフェーズでは、余裕がある場合にvelocity推定アルゴリズムを用いた強弱制御機能を追加実装する。これにより、指揮棒を振る強さによって音量が変化する、より表現力の高い演奏システムを目指す。また、本ガントチャートでは複数の作業を並列に実施することで、全体の開発期間短縮を図っている。特に、音色生成系と通信系を同時進行で開発することで、効率的なシステム開発を実現する計画としている。

12 検証と妥当性確認: V&V 計画

12-1. プロジェクトの成果物に対する有効指標 MOE

有効指標 (MOE) は、システムが最終的にユーザー (指揮者・聴衆) に対して提供する価値を評価する指標である。本プロジェクトでは、各テスト階層における性能指標 (MOP) の達成を通じて、以下の3つの有効性を確立する。

表 4 MOE と MOP の対応関係および目標値

有効指標 (MOE)	性能指標 (MOP)	目標値 / 達成水準
MOE 1: 音楽的再現性と正確性 (単体テスト領域)	拍検出の正確性	正解率 $\geq 90\%$
	音階の誤差	セント単位での許容範囲内
	楽譜との相違	誤ノート発音数 0
MOE 2: 合奏の一体感とインタラクティブ性 (結合テスト領域)	楽器間同期誤差	≤ 20 ms
	指揮から楽器への通信遅延	≤ 30 ms
	テンポ追従の遅延	≤ 2 拍
MOE 3: システムとしての堅牢性と実用性 (システムテスト領域)	起動時間	≤ 5 s
	CPU 負荷 (入力フェーズ)	≤ 2 ms
	パケットロスの許容範囲	演奏継続が可能なレベル

12-2. 有効指標 MOE に対応する性能指標 MOP

前項で定義した各 MOE に対し、システム全体として達成すべき性能指標を以下の通り定める。これらは、特定の検証フェーズに限定されず、成果物が最終的に備えるべき技術的到達基準である。

- **音楽的品質の達成基準**

システムが「音楽」として機能するための基準である。指揮動作から拍を正確に解析し、指定された楽譜データを正確なピッチで物理的な音響・動作へと変換する精度を評価の主軸とする。

- **リアルタイム・アンサンブルの達成基準**

システムが「合奏」および「指揮」として機能するための基準である。分散配置された複数ノード間での極小な同期誤差の維持、および指揮者の動的な変化（テンポ変化等）に対する停滞のない追従性能を評価の主軸とする。

- **運用プラットフォームとしての達成基準**

システムが「実用的な道具」として機能するための基準である。限られた計算資源内での処理完結、無線環境下での通信断絶耐性、および即時稼働を可能にする運用性を評価の主軸とする。

12-3. システムテストで評価する性能指標 MOP

パケットロスの許容範囲は聴感破綻の発生が起きない 5% 以内のパケットロスを指標とする。CPU の負荷については、入力装置をしてから 2ms 以内を指標とする。

12-4. 結合テストで評価する性能指標 MOP

通信の同期を行う際、20ms 以内のズレであった場合は 2 つの音を別々に認識できないため、同期の誤差を 20ms 以内を指標とする [?]. 指揮棒から楽器への通信の遅延については、10ms 以内を指標とする。テンポの追従の遅延については、2 拍以内を指標とする。

12-5. 単体テストで評価する性能指標 MOP

拍検知の正確性については、90% 以上を指標とする。音階の誤差については、自己相関による音階解析を行った際の、平均値が 5.52cent であったため、5.52cent までの誤差を指標とする [?].

12-6. プロジェクト中に監視する技術性能指標 TPM

本項では、開発期間中において各性能指標の達成見込みを継続的に評価するための技術性能指標 (TPM) を定義する。これらは、設計変更や機能実装に伴う技術的リスクを早期に発見することを目的として監視を行う。

- **同期誤差の推移監視**

最終目標値である 20 ms に対する現在の到達度を監視する。特に通信プロトコルの最適化やノード数の増減が発生した際に実測を行い、目標値からの乖離が 10 ms 以上となった場合には、バッファ制御アルゴリズムの再検討を実施する。

- **CPU 処理時間のマージン監視**

制御周期 5 ms に対する各タスクの処理時間を監視する。機能追加に伴い CPU 負荷が目標値 (2 ms) の 80 % を超過した段階で、コードの最適化およびタスク優先順位の再設計を行い、リアルタイム性の破綻を未然に防止する。

12-7. プロジェクト中に監視する指標 TPM

技術的な数値指標に加え、プロジェクト全体の各システム性能を維持するために、リソースおよび進捗の観点から以下の指標を監視する。

- **機能実装の完了率**

WBS に基づく各モジュールの実装進捗を監視する。特に指揮解析と通信基盤の統合が予定より遅延した場合は、クリティカルパス上のリスクとして要員配置の再検討やスコープの調整を行う。

- **システム稼働の安定度**

開発中の連続稼働テストにおける意図しないリセットやハングアップの発生頻度を監視する。ハードウェア起因の不安定要素やメモリリークを早期に特定・排除し、最終的なシステムテストにおける信頼性を担保する。

13 資源・調達管理

本節では、本プロジェクトの遂行に必要となるハードウェア資源および開発環境の調達計画について述べる。

- **ハードウェア資源**

本プロジェクトでは、計 5 台の Arduino (指揮用 1 台, 楽器用 4 台) を使用する。また、指揮解析用の IMU センサおよび各楽器ノードの出力用デバイスについても、チーム内の既存資産および新規調達によって確保済みである。

- **開発・運用リソース**

ソースコードの管理には Git を用い、チーム内での並行開発を可能とする。また、無線通信環境の検証用として専用のルーターを用い、ハッカソン会場等の外部環境に依存しない安定した通信経路を確保する。

14 リスク管理

それぞれの担当箇所の作業日を長めにとることで、病気や、不具合が発生し作業が滞ったとしても、余分に作業日を設けているのでスケジュール通りの作業に修正することが容易になる。